

Graduate Diploma in Systems Analysis (GDipSA)

[A. Industrial Attachment Programme](#)

[B. Background of GDipSA](#)

A. Industrial Attachment Programme

The incoming Industrial Attachment (IA) programme will be from **7th September 2026 to 22nd January 2027**.

1. This industrial attachment is a credit-bearing module that will contribute towards the student's graduation from the course.
2. The project(s) should cover phases of the software development life cycle with a **minimum of 40% of coding assignments**. The students will be required to present their IA project to NUS-ISS staff for assessment and to an Accenture partner if the team is shortlisted to vie for the Accenture Prize.
3. The recommended allowance for each student is \$1000-1500 per month.
4. The organisation has to provide the required resources for the students to undertake the project at its site and to assign one of your IT personnel to supervise them. At the end of the project, the organisation supervisor will be asked to provide feedback on the quality of the project deliverables and performance of the students.
5. Once the project is confirmed, we would prefer if the organisation does not change the project, alter its scope or the composition of the IA team.
6. Please visit the [program website](#) for more information.

B. Background of GDipSA

Purpose of GDipSA

[GDipSA](#) is a twelve-month full-time programme designed to convert non-IT graduates into IT-savvy professionals since 1981. In addition to state-of-the-art technical topics, the business-oriented nature of the curriculum equips the graduates with an understanding of the role of information systems in business and government organisations.

Course Structure (Lectures, Workshops, Labs, Industrial Attachment)

Web Application Development

Programming Fundamentals (Java): Core programming concepts and problem-solving using Java.

Object-Oriented Programming (Java): OOP principles and advanced concepts (e.g., inheritance, polymorphism, encapsulation).

Relational Databases & SQL: Database fundamentals, SQL querying, and working with relational database management systems (RDBMS).

Internet & Web Technologies: Web foundations including HTTP/HTTPS, client-server architecture, and web standards.

Front-end Web Development: Building responsive and interactive web interfaces using HTML, CSS, Bootstrap, JavaScript, and Angular, including component-based UI development.

Back-end Web Development (Spring Boot): Developing server-side applications using Spring Boot, including MVC, RESTful services, and layered application architecture.

Persistence & ORM: Implementing persistence using JPA with Hibernate as the ORM provider; designing repository layers.

Data Access: Writing queries using JPQL; using Spring Data for repository abstraction; introduction to NoSQL access via Spring Data extensions.

Enterprise Java EE Web Development: Designing and developing enterprise-grade web applications using Java EE concepts and best practices (e.g., resource management, connection pooling).

Cloud Deployment (AWS): Deploying and running applications on AWS services.

Designing Digital Solutions

Requirements Elicitation: Techniques to gather, validate, and document stakeholder and user needs.

UI/UX Design: User-centred design principles, wireframing/prototyping, and usability considerations.

Solution Scoping: Translating business needs into solution scope, constraints, assumptions, and high-level design.

Back-end / System Design: Designing back-end components and services for distributed systems.

Software Analysis & Design (UML & Patterns):

System Analysis: Analysing requirements, constraints, and non-functional considerations.

Requirements Modelling: Use cases and/or user stories to define functional requirements.

UML Modelling: Class and sequence diagrams to represent structure and interactions.

Design Patterns: Applying common patterns to improve maintainability and scalability.

Architectural Styles: Comparing styles such as MVC, layered architecture, and service-based designs.

Developing Mobile Solutions

Android Development (Kotlin): Android application development using Kotlin and core Android frameworks.

Mobile App Implementation: Building, testing, and packaging applications using Android Studio.

Mobile UI Development: Designing reusable UI components and implementing responsive mobile interfaces.

On-device Data Storage: Implementing local storage options.

Full-stack Integration: Integrating mobile apps with back-end services (e.g., Spring Boot REST APIs and/or ASP.NET MVC/.NET services, depending on project requirements).

DevSecOps Engineering and Automation

DevSecOps Fundamentals: Integrating security practices into the DevOps lifecycle and delivery pipeline.

Agile Software Delivery: Applying Agile practices for iterative development and delivery.

Continuous Integration (CI): Automated build and test workflows to ensure code quality and early defect detection.

Continuous Delivery/Deployment (CD): Automated release pipelines to deploy changes reliably to staging/production environments.

CI/CD Security: Incorporating security controls into pipelines (e.g., dependency checks, secrets detection, static analysis where applicable).

Security Automation & Compliance: Automating security checks, policy enforcement, and audit-ready practices within the SDLC.

Developing Machine Learning Solutions

Data Analysis with Python: Using NumPy, pandas, Matplotlib, and Seaborn for data preparation, exploration, and visualisation.

API Development (Python): Building and publishing services as RESTful APIs in Python.

Machine Learning Modelling: Implementing common ML approaches (e.g., k-NN, Logistic Regression, k-Means, Neural Networks, and Time Series forecasting/modelling).

Model Training & Evaluation: Training models and evaluating performance using appropriate metrics and validation approaches.

Feature Engineering & Dimensionality Reduction: Applying feature selection/engineering techniques (e.g., correlation analysis) and dimensionality reduction methods such as PCA.

Developing Agentic AI Solutions

Large Language Models (LLMs): Prompt engineering and development of Retrieval-Augmented Generation (RAG) workflows to integrate generative AI capabilities into software solutions.

Agentic System Design: Designing and developing single-agent and multi-agent systems for task planning and execution.

Tools & Guardrails: Implementing tool use, planning strategies, safety/constraint guardrails, and evaluation considerations.